

ABITIA — PLAN DE PROYECTO TÉCNICO

Estructura de Conversión de Sistema de Condominio a Web Local-First / Google Antigravity

Proyecto:	Abitia (Gestión Integral de Multi-Condominios)	Fecha de Emisión:	8 de Junio de 2026
Fase Actual:	Fase Magra (MVP Estricto de Alta Densidad)	Entorno Destino:	Google Antigravity & MySQL Local Core
Dominios Activos:	abitia.co (Corporativo/Landing) / abitia.app (Operativo/HTTPS HSTS)	Filosofía:	Unix-Minded, Autonomía Local, Cero Desperdicio Operativo

1. Premisas Analizadas y Premisas de Control

El proyecto **Abitia** representa la transformación radical de un sistema clásico de escritorio a un ecosistema web distribuido y resiliente. Basado en las deliberaciones previas, el sistema se fundamenta en las siguientes premisas estratégicas:

- **Estrategia de Doble Dominio Alternado:** Segmentación estricta de responsabilidades entre `abitia.co` (puerta comercial) y `abitia.app` (entorno seguro transaccional).
- **Aislamiento Multi-Tenant Integrado:** Una infraestructura de base de datos única que utiliza identificadores de inquilinos (`IdCondominio`) indexados para impedir el cruce accidental de registros financieros.
- **Arquitectura de Contabilidad Ledger Inmutable:** Exclusión de cálculos dinámicos acumulativos en caliente. Cada movimiento de deuda o crédito genera un registro permanente e irreversible para auditorías exactas.
- **Verificación Asíncrona de Fondos:** Los pagos reportados por los copropietarios entran en una zona de contención previa antes de ser validados de manera manual por la junta administradora, blindando el Ledger contra fraudes bancarios.
- **Diseño de Interfaces de Alta Densidad:** Maximización de la información expuesta por pantalla para emular la agilidad de los sistemas desktop clásicos, adaptados a flujos rápidos de datos.

2. Propuestas Técnicas de Control (Antes de Iniciar)

Propuesta 1: Compatibilidad estricta con Google Antigravity

Google Antigravity exige empaquetados altamente desacoplados, autónomos y preparados para correr de forma "Local-First". Para cumplir este estándar, la capa de acceso a datos de Abitia debe operar mediante inyección de dependencias estricta. Las consultas SQL no se incrustarán en las vistas, sino que se aislarán en servicios puros que puedan interactuar tanto con motores MySQL locales como remotos a través de búferes de memoria persistentes.

Propuesta 2: El Enfoque Híbrido Previsor del Fondo de Reserva

A pesar de encontrarnos en una Fase Magra donde el cálculo dinámico de fondos está deshabilitado, las tablas base incluirán las columnas estructurales de reserva configuradas con valores por defecto (0.00). Esto asegura que cuando se active la Fase 2, la integración consistirá únicamente en alterar la regla aritmética del backend sin requerir migraciones estructurales destructivas (*Zero-Downtime Migration*).

3. Script Automatizado de Base de Datos (MySQL)

A continuación se detalla el archivo DDL para MySQL. El script incluye restricciones de llave foránea, índices compuestos óptimos para búsquedas masivas bajo entornos Multi-Tenant, y motores de almacenamiento de alta transaccionalidad (InnoDB).

```

-- =====
-- SCRIPT DE CREACIÓN DE BASE DE DATOS - PROYECTO ABITIA (FASE MAGRA)
-- MOTOR: MySQL 8.0+ / InnoDB
-- =====

CREATE DATABASE IF NOT EXISTS abitia_core CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE abitia_core;

-- 1. TABLA NÚCLEO: CONDOMINIO (TENANTS)
CREATE TABLE Condominio (
    IdCondominio INT AUTO_INCREMENT PRIMARY KEY,
    Nombre VARCHAR(120) NOT NULL,
    Rif_IdFiscal VARCHAR(50) NOT NULL UNIQUE,
    Direccion TEXT NOT NULL,
    Subdominio_Slug VARCHAR(50) NOT NULL UNIQUE,
    Porcentaje_Fondo_Reserva DECIMAL(5,2) NOT NULL DEFAULT 0.00, -- Columna previsoras Fase 2
    Configuracion_JSON JSON NULL,
    Fecha_Creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. TABLA: USUARIO GLOBAL
CREATE TABLE Usuario (
    IdUsuario INT AUTO_INCREMENT PRIMARY KEY,
    Nombre VARCHAR(80) NOT NULL,
    Apellido VARCHAR(80) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Password_Hash VARCHAR(255) NOT NULL,
    Telefono VARCHAR(30) NULL,
    Estatus TINYINT NOT NULL DEFAULT 1, -- 1: Activo, 0: Inactivo
    Fecha_Registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 3. TABLA DE INTERSECCIÓN: ROLES MULTI-CONDOMINIO
CREATE TABLE Usuario_Condominio_Rol (
    IdUsuario INT NOT NULL,
    IdCondominio INT NOT NULL,
    Rol TINYINT NOT NULL, -- 1: SuperAdmin, 2: Administrador, 3: Propietario, 4: Inquilino
    PRIMARY KEY (IdUsuario, IdCondominio),
    CONSTRAINT FK_Rol_Usuario FOREIGN KEY (IdUsuario) REFERENCES Usuario(IdUsuario) ON DELETE
    CASCADE,
    CONSTRAINT FK_Rol_Condominio FOREIGN KEY (IdCondominio) REFERENCES Condominio(IdCondominio)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- 4. TABLA: PROPIEDAD (INMUEBLES)
CREATE TABLE Propiedad (
    IdPropiedad INT AUTO_INCREMENT PRIMARY KEY,
    IdCondominio INT NOT NULL,
   Codigo_Nro VARCHAR(30) NOT NULL, -- Ej: 'APT-10A', 'CASA-12'
    Alicuota DECIMAL(10,6) NOT NULL, -- Porcentaje de participación, ej: 0.041250
    IdPropietario_Actual INT NOT NULL,
    Estatus TINYINT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Propiedad_Condominio FOREIGN KEY (IdCondominio) REFERENCES
    Condominio(IdCondominio),
    CONSTRAINT FK_Propiedad_Propietario FOREIGN KEY (IdPropietario_Actual) REFERENCES
    Usuario(IdUsuario),
    UNIQUE KEY UX_Condominio_Inmueble (IdCondominio, Codigo_Nro)
) ENGINE=InnoDB;

-- 5. TABLA: GASTO_CONDOMINIO (EGRESOS MENSUALES)
CREATE TABLE Gasto_Condominio (

```

```

    IdGasto INT AUTO_INCREMENT PRIMARY KEY,
    IdCondominio INT NOT NULL,
    Periodo_MesAnio VARCHAR(7) NOT NULL, -- Formato: 'YYYY-MM'
    Descripcion VARCHAR(255) NOT NULL,
    Monto DECIMAL(18,2) NOT NULL,
    Tipo_Gasto TINYINT NOT NULL, -- 1: Común (Por Alícuota), 2: No Común (Cargado directo)
    Fecha_Gasto DATE NOT NULL,
    CONSTRAINT FK_Gasto_Condominio FOREIGN KEY (IdCondominio) REFERENCES
Condominio(IdCondominio)
) ENGINE=InnoDB;

-- 6. TABLA: RECIBO_MENSUAL (CUOTAS GENERADAS)
CREATE TABLE Recibo_Mensual (
    IdRecibo INT AUTO_INCREMENT PRIMARY KEY,
    IdPropiedad INT NOT NULL,
    Periodo_MesAnio VARCHAR(7) NOT NULL,
    Fecha_Emission DATE NOT NULL,
    Fecha_Vencimiento DATE NOT NULL,
    Monto_Gasto_Comun DECIMAL(18,2) NOT NULL,
    Monto_Gasto_NoComun DECIMAL(18,2) NOT NULL DEFAULT 0.00,
    Monto_Fondo_Reserva DECIMAL(18,2) NOT NULL DEFAULT 0.00, -- Columna previsoras Fase 2
    Saldo_Anterior_Pendiente DECIMAL(18,2) NOT NULL DEFAULT 0.00,
    Total_A_Pagar DECIMAL(18,2) NOT NULL,
    Estatus_Pago TINYINT NOT NULL DEFAULT 0, -- 0: Pendiente, 1: Pagado, 2: Parcial
    CONSTRAINT FK_Recibo_Propiedad FOREIGN KEY (IdPropiedad) REFERENCES Propiedad(IdPropiedad)
) ENGINE=InnoDB;

-- 7. TABLA DE CONTENCIÓN: PAGO_REPORTADO (VERIFICACIÓN ASÍNCRONA)
CREATE TABLE Pago_Reportado (
    IdPago BIGINT AUTO_INCREMENT PRIMARY KEY,
    IdCondominio INT NOT NULL,
    IdPropiedad INT NOT NULL,
    IdUsuario_Reporta INT NOT NULL,
    Fecha_Reporte TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Monto DECIMAL(18,2) NOT NULL,
    Fecha_Transferencia DATE NOT NULL,
    Referencia_Bancaria VARCHAR(50) NOT NULL,
    IdBanco_Destino INT NOT NULL,
    Forma_Pago TINYINT NOT NULL, -- 1: Transferencia, 2: Pago Móvil, 3: Efectivo, 4: Zelle
    Comprobante_Url VARCHAR(255) NULL,
    Observaciones_User VARCHAR(255) NULL,
    Estatus_Verificacion TINYINT NOT NULL DEFAULT 0, -- 0: Pendiente, 1: Aprobado, 2: Rechazado
    Fecha_Verificacion DATETIME NULL,
    IdUsuario_Verifica INT NULL,
    Motivo_Rechazo VARCHAR(255) NULL,
    CONSTRAINT FK_Pago_Condominio FOREIGN KEY (IdCondominio) REFERENCES
Condominio(IdCondominio),
    CONSTRAINT FK_Pago_Propiedad FOREIGN KEY (IdPropiedad) REFERENCES Propiedad(IdPropiedad),
    CONSTRAINT FK_Pago_Usuario FOREIGN KEY (IdUsuario_Reporta) REFERENCES Usuario(IdUsuario),
    CONSTRAINT FK_Pago_Admin FOREIGN KEY (IdUsuario_Verifica) REFERENCES Usuario(IdUsuario)
) ENGINE=InnoDB;

-- 8. TABLA CONTABLE NÚCLEO: CUENTA_CORRIENTE_PROPIEDAD (LEDGER INMUTABLE)
CREATE TABLE Cuenta_Corriente_Propiedad (
    IdMovimiento BIGINT AUTO_INCREMENT PRIMARY KEY,
    IdCondominio INT NOT NULL,
    IdPropiedad INT NOT NULL,
    Fecha_Movimiento DATETIME NOT NULL,
    Tipo_Movimiento TINYINT NOT NULL, -- 1: Recibo Emitido, 2: Pago Aplicado, 3: Nota Crédito,
4: Nota Débito
    IdReferencia_Origen INT NOT NULL, -- Enlace al IdRecibo o IdPago
    Descripcion VARCHAR(150) NOT NULL,

```

```

Monto DECIMAL(18,2) NOT NULL,
Saldo_Resultante DECIMAL(18,2) NOT NULL,
CONSTRAINT FK_Ledger_Condominio FOREIGN KEY (IdCondominio) REFERENCES
Condominio(IdCondominio),
CONSTRAINT FK_Ledger_Propiedad FOREIGN KEY (IdPropiedad) REFERENCES Propiedad(IdPropiedad)
) ENGINE=InnoDB;

-- =====
-- ÍNDICES COMPUESTOS CRÍTICOS PARA ALTA DENSIDAD Y ESCALA MULTI-TENANT
-- =====
CREATE INDEX IX_Condominio_Slug ON Condominio (Subdominio_Slug);
CREATE INDEX IX_Propiedad_FiltroCore ON Propiedad (IdCondominio, IdPropiedad);
CREATE INDEX IX_Gasto_Mes ON Gasto_Condominio (IdCondominio, Periodo_MesAnio);
CREATE INDEX IX_Recibo_Pendiente ON Recibo_Mensual (IdPropiedad, Estatus_Pago);
CREATE INDEX IX_Pago_BandejaVerificar ON Pago_Reportado (IdCondominio, Estatus_Verificacion,
Fecha_Reporte DESC);
CREATE UNIQUE INDEX UX_Pago_PreencionFraude ON Pago_Reportado (IdCondominio,
Referencia_Bancaria);

-- Índice de lectura instantánea para estados de cuenta de alta velocidad
CREATE INDEX IX_Ledger_Historico Instantaneo ON Cuenta_Corriente_Propiedad (IdCondominio,
IdPropiedad, IdMovimiento DESC);

```

4. Procedimientos de Operación de Dominios y Multi-Tenancy

El manejo de flujos e infraestructura web en Abitia se rige por un esquema distribuido acoplado a la perfección con la lógica de servidores requerida por Google Antigravity.

4.1 Arquitectura del Ruteo de Peticiones

Para procesar el tráfico procedente de los dos dominios adquiridos, se implementa un proxy inverso local de configuración estricta. El comportamiento de las llamadas se divide según su origen:

Dominio Web / URL	Capa del Servidor	Acción del Backend / Inyección	Acceso Base de Datos
abitia.co (y www)	Página Pública Estática	Entrega contenido de marketing, planes y captación.	Ninguno (Cero persistencia de negocio).
app.abitia.app	Controlador de Acceso Global	Despliega login general si no se especifica condominio.	Lectura global de credenciales de Usuario.
[slug].abitia.app	Middleware Multi-Tenant	Extrae el subdominio, resuelve el ID y ancla la sesión.	Filtro exclusivo inyectado por IdCondominio.

4.2 Lógica Interna de Conexión y Resolución en Código

Cuando se procesa una petición bajo el subdominio `elparaíso.abitia.app`, el componente distribuidor ejecuta la siguiente subrutina lógica:

- Aislamiento de Cabecera:** Se extrae el fragmento del subdominio de la cabecera HTTP Host mediante operaciones lógicas de strings de bajo nivel (*parsing* limpio).
- Búsqueda en Caché de Contexto:** Se verifica si el slug extraído (ej: "elparaíso") se encuentra precargado en la memoria segura del servidor. Si existe, se recupera de manera inmediata el `IdCondominio` asociado.
- Inyección Forzada:** Todas las sentencias y comandos SQL ejecutados en ese ciclo de vida del hilo de ejecución anexan de manera obligatoria el filtro de seguridad indexado:

Query = Base_Query + " WHERE IdCondominio = " + CurrentTenantId

5. El Workflow Matemático del Ledger Magro

Al procesarse las transacciones financieras dentro de Abitia, la matemática sigue una secuencia lineal e incremental para preservar la integridad de los balances.

Fórmula de Emisión de Recibo Mensual:

Para cada inmueble perteneciente al inquilino actual, el monto del gasto común se define como la sumatoria total de los egresos multiplicados por la alícuota legal asignada:

$$\text{Monto_Gasto_Comun} = \left(\sum_{i=1}^n \text{Gasto_Comun}_i \right) \text{ imes Alicuota}$$

El valor de Total_A_Pagar se compone de los gastos comunes, los cargos directos no comunes (como llaves o reparaciones particulares), el fondo de reserva previsor (fijado en 0.00 para la fase magra) y el arrastre extraído del Ledger:

$$\text{Total_A_Pagar} = \text{Monto_Gasto_Comun} + \text{Monto_Gasto_NoComun} + 0.00 + \text{Saldo_Anterior_Pendiente}$$

Alineación de Auditoría Inmutable:

- **Inserción Inicial:** Al confirmarse el recibo de cobro, la tabla Cuenta_Corriente_Propiedad agrega un registro de tipo 1. El nuevo saldo es la suma exacta de la deuda saliente del ledger más el monto actual del recibo.
- **Abonos de Tesorería:** Cuando un pago transita de la tabla Pago_Reportado al estado aprobado (Estatus = 1), se inyecta un registro contable tipo 2. El saldo deudor neto del propietario se reduce restando el importe verificado directamente del balance resultante previo, manteniendo la traza histórica impecable y libre de recálculos destructivos.